ENTERPRISE TECHNICAL SPECIFICATION

Version 0.2.0
Author: Ansh Rajore

AEONRIFT

Time-Travel Infrastructure & Universal Model Context Protocol (MCP) Runtime

Official Production URL:		https://aeonrift.vercel.app
Public NPM Package:	https://www.npmjs.com/package/aeonrift	(v0.2.0)
GitHub Repository:		AEONRIFT on GitHub
Supported Runtimes:	Claude Desktop, Cursor IDE, Windsurf, LangChain, CrewAI, AutoGen, Node.js, Python 3.8+	

1. What Is AEONRIFT?

When an AI agent crashes mid-task, typical systems force a complete, blind restart. In enterprise environments, this causes duplicate Stripe credit charges, repeated database mutations, and wasted tokens.

AEONRIFT is a high-throughput time-travel runtime that provides mathematical causal durability:

- **SHA-256 Causal State DAG**: Tracks causal lineage across all LLM steps and tool calls.
- **ACRFence Rollback Guard**: Prevents semantic duplicate side-effects (e.g. charging a credit card twice).
- **Multi-Level Checkpoint Hierarchy (L0–L5)**: Prunes wasteful turn-based snapshots by >75% based on 2026 Crab research.
- **High-Throughput Ring Buffer & Delta Compressor**: Handles 50,000 events/sec with 70–85% payload compression.

2. Quick Start: Universal Model Context Protocol (MCP)

Any autonomous LLM agent or IDE (Claude Desktop, Cursor, Windsurf) can connect to AEONRIFT with zero custom glue code.

Step 1: Verify Installation

```
# Test the engine and diagnostics
python3 ./aeonrift doctor
python3 ./aeonrift mcp
```

Step 2: Connect Claude Desktop

> Add the following block to your `claude_desktop_config.json`:
>
> ```
> {
>   "mcpServers": {
>     "aeonrift": {
>       "command": "python3",
>       "args": ["-m", "aeonrift.cli.main", "mcp"],
>       "env": {
>         "PYTHONPATH": "/absolute/path/to/AEONRIFT"
>       }
>     }
>   }
> }
> ```

> ### Step 3: Connect Cursor IDE
>
> Open **Cursor Settings > Features > MCP** and click **Add New MCP Server**:
>
> - **Name**: `aeonrift`
> - **Type**: `command`
> - **Command**: `python3 ./aeonrift mcp`

3. Available MCP Enterprise Tools

MCP Tool Name	Functionality	Enterprise Value
`aeonrift_record_event`	Intercepts tool calls and database writes into the causal DAG.	Immutable, tamper-evident audit trail of every agent action.
`aeonrift_check_rollback_hazard`	Verifies whether replaying an action causes duplicate mutations.	Prevents duplicate payments and ACRFence security vulnerabilities.
`aeonrift_create_checkpoint`	Creates HMAC-signed snapshot of agent variables and memory.	Enables sub-second state rewinds.
`aeonrift_autonomous_recover`	Analyzes failure nodes and computes optimal recovery plans.	Autonomous self-healing without human intervention.
`aeonrift_inspect_timeline`	Returns full step-by-step causal execution lineage.	Deep debugging and multi-agent coordination visibility.
`aeonrift_diff_checkpoints`	Calculates delta dictionary between two checkpoints.	Zero-overhead state diffing.
`aeonrift_health_doctor`	Real-time engine and buffer health diagnostic probe.	Production telemetry and cluster heartbeat monitoring.

4. CLI Commands Reference (macOS, Linux, Windows)

Command	Syntax	Description
`mcp`	`aeonrift mcp`	Launches standard Model Context Protocol server over stdio.
`doctor`	`aeonrift doctor`	Validates Python runtime, DAG engine, and ledger health.
`benchmark`	`aeonrift benchmark`	Runs the RIFT-Bench evaluation suite against agent workloads.
`timeline`	`aeonrift timeline <exec_id>`	Displays ASCII/UTF-8 formatted timeline of execution events.
`export`	`aeonrift export <id> --format dot`	Exports graph to Graphviz DOT or raw JSON.
`replay`	`aeonrift replay <exec_id>`	Interactive time-travel replay runner with step-through mode.

5. Published NPM Package Usage

```
npm install aeonrift@0.2.0
```

```
import { AeonriftMCPBridge } from "aeonrift";

const bridge = new AeonriftMCPBridge("session_alpha");
const tools = bridge.getAvailableTools();

// Automatically handle agent actions
const result = bridge.handleToolCall("aeonrift_record_event", {
  tool_name: "stripe_payment",
  arguments: { amount: 150 }
});
```
